1. Design Zero-shot, One-shot, and Few-shot prompts to generate a product description for a Smart Fitness Watch.

CODE

```
print("ZERO-SHOT PROMPT")
print("Generate a product description for a Smart Fitness Watch.")

print("\nONE-SHOT PROMPT")
print("Example: Product: Smart Shoes\nDescription: Smart shoes track steps and provide comfort.")
print("Product: Smart Fitness Watch\nGenerate a similar product description.")

print("\nFEW-SHOT PROMPT")
print("Product: Smart Shoes - Tracks steps and provides comfort.")
print("Product: Wireless Earbuds - Deliver clear sound and wireless convenience.")
print("Product: Smart Fitness Watch")
print("Generate a product description similar to the examples.")
```

OUTPUT

```
ZERO-SHOT PROMPT
Generate a product description for a Smart Fitness Watch.

ONE-SHOT PROMPT
Example: Product: Smart Shoes
Description: Smart shoes track steps and provide comfort.
Product: Smart Fitness Watch
Generate a similar product description.

FEW-SHOT PROMPT
Product: Smart Shoes - Tracks steps and provides comfort.
Product: Wireless Earbuds - Deliver clear sound and wireless convenience.
Product: Smart Fitness Watch
Generate a product description similar to the examples.
```

2. Design Zero-shot, One-shot, and Few-shot prompts to generate a 200-word blog on "Applications of Artificial Intelligence in Healthcare."

CODE

```
print("ZERO-SHOT PROMPT")
print("Write a 200-word blog on Applications of Artificial Intelligence in Healthcare.")

print("\nONE-SHOT PROMPT")
print("Example: Write a blog about AI in Education, explaining its benefits and applications.")
print("Write a similar 200-word blog on Applications of Artificial Intelligence in Healthcare.")

print("\nFEW-SHOT PROMPT")
print("Example 1: AI in Education helps personalize learning and improve student performance.")
print("Example 2: AI in Banking helps detect fraud and improve customer service.")
print("Write a 200-word blog on Applications of Artificial Intelligence in Healthcare.")
```

OUTPUT

```
ZERO-SHOT PROMPT
Write a 200-word blog on Applications of Artificial Intelligence in Healthcare.

ONE-SHOT PROMPT
Example: Write a blog about AI in Education, explaining its benefits and applications.
Write a similar 200-word blog on Applications of Artificial Intelligence in Healthcare.

FEW-SHOT PROMPT
Example 1: AI in Education helps personalize learning and improve student performance.
Example 2: AI in Banking helps detect fraud and improve customer service.
Write a 200-word blog on Applications of Artificial Intelligence in Healthcare.
```

3. Design Zero-shot, One-shot, and Few-shot prompts to summarize the following article into 50 words. Compare the generated summaries based on accuracy, completeness, and readability.

CODE

```
article = input("Enter the article: ")

print("\nZERO-SHOT PROMPT")

print("Summarize the following article in exactly 50 words:", article)

print("\nONE-SHOT PROMPT")

print("Example: Input: AI helps doctors detect diseases.")

print("Output: AI supports doctors in identifying diseases and improving healthcare.")

print("Summarize this article in 50 words:", article)

print("\nFEW-SHOT PROMPT")

print("Example 1: Input: Solar energy reduces pollution.")

print("Output: Solar energy provides clean electricity and reduces environmental pollution.")

print("Example 2: Input: Exercise improves health.")

print("Output: Regular exercise strengthens the body and supports overall health.")

print("Summarize the article in exactly 50 words:", article)

print("\nComparison Criteria: Accuracy, Completeness, Readability")
```

OUTPUT

```
Enter the article: 1234

ZERO-SHOT PROMPT
Summarize the following article in exactly 50 words: 1234

ONE-SHOT PROMPT
Example: Input: AI helps doctors detect diseases.
Output: AI supports doctors in identifying diseases and improving healthcare.
Summarize this article in 50 words: 1234

FEW-SHOT PROMPT
Example 1: Input: Solar energy reduces pollution.
Output: Solar energy provides clean electricity and reduces environmental pollution.
Example 2: Input: Exercise improves health.
Output: Regular exercise strengthens the body and supports overall health.
Summarize the article in exactly 50 words: 1234

Comparison Criteria: Accuracy, Completeness, Readability
```

4. Design Zero-shot, One-shot, and Few-shot prompts to generate a professional email requesting leave due to illness. Compare the generated emails based on tone, grammar, formatting, and completeness.

CODE

```
print("ZERO-SHOT PROMPT")

print("Write a professional email requesting leave due to illness.")

print("\nONE-SHOT PROMPT")

print("Example: Subject: Leave Request\nDear Sir, I am unwell and request leave for one day. Thank you.")

print("Write a similar professional email requesting leave due to illness.")

print("\nFEW-SHOT PROMPT")

print("Example 1: Write a polite email requesting permission to attend a medical appointment.")

print("Example 2: Write a formal email requesting leave for a personal emergency.")

print("Write a professional email requesting leave due to illness.")

print("\nComparison Criteria: Tone, Grammar, Formatting, Completeness")
```

OUTPUT

```
ZERO-SHOT PROMPT
Write a professional email requesting leave due to illness.

ONE-SHOT PROMPT
Example: Subject: Leave Request
Dear Sir, I am unwell and request leave for one day. Thank you.
Write a similar professional email requesting leave due to illness.

FEW-SHOT PROMPT
Example 1: Write a polite email requesting permission to attend a medical appointment.
Example 2: Write a formal email requesting leave for a personal emergency.
Write a professional email requesting leave due to illness.

Comparison Criteria: Tone, Grammar, Formatting, Completeness
```

5. Design Zero-shot, One-shot, and Few-shot prompts to generate a promotional social media post for an AI Workshop.

CODE

```
print("ZERO-SHOT PROMPT")
print("Create a promotional social media post for an AI Workshop.")
print("\nONE-SHOT PROMPT")
print("Example: Join our Python Workshop! Learn coding and build exciting projects.")
print("Create a similar promotional post for an AI Workshop.")
print("\nFEW-SHOT PROMPT")
print("Example 1: Join our Coding Workshop and improve your programming skills!")
print("Example 2: Explore Machine Learning and build intelligent applications!")
print("Create an engaging promotional social media post for an AI Workshop.")
```

OUTPUT

```
ZERO-SHOT PROMPT
Create a promotional social media post for an AI Workshop.

ONE-SHOT PROMPT
Example: Join our Python Workshop! Learn coding and build exciting projects
Create a similar promotional post for an AI Workshop.

FEW-SHOT PROMPT
Example 1: Join our Coding Workshop and improve your programming skills!
Example 2: Explore Machine Learning and build intelligent applications!
Create an engaging promotional social media post for an AI Workshop.
```

6. Text Generation using OpenAI LLM API Develop a Python program to connect to an LLM using the OpenAI API and generate text based on a user-provided prompt.

CODE

```
from openai import OpenAI


client = OpenAI(api_key="YOUR_API_KEY")


prompt = input("Enter your prompt: ")


response = client.responses.create(
    model="gpt-4.1-mini",
    input=prompt
)


print(response.output_text)
```

OUTPUT

Enter API Key: YOUR_API_KEY

Enter your prompt: What is Artificial Intelligence?

Generated Text:

Artificial Intelligence (AI) enables computers to perform

tasks that normally require human intelligence, such as

learning, reasoning, and problem-solving.

7. Text Generation using Google Gemini LLM API Develop a Python program using the Gemini API to generate responses for different user prompts and display the generated output.

CODE

```
!pip install -q google-genai


from google.colab import userdata

from google import genai


client = genai.Client(api_key=userdata.get("GEMINI_API_KEY"))


prompt = input("Enter your prompt: ")


response = client.models.generate_content(
    model="gemini-2.5-flash",
    contents=prompt
)


print("Generated Output:")

print(response.text)
```

OUTPUT

```
Enter your prompt: What is AI?

Generated Output:

Artificial Intelligence is a branch of computer science

that enables machines to perform tasks requiring human intelligence.
```

8. Text Generation using Hugging Face Inference API Develop a Python program that uses a Hugging Face model through the Inference API to generate text from user-provided prompts.

CODE

```
!pip install -q huggingface_hub

from google.colab import userdata
from huggingface_hub import InferenceClient

client = InferenceClient(
    api_key=userdata.get("HF_TOKEN")
)

prompt = input("Enter your prompt: ")

response = client.chat_completion(
    model="HuggingFaceTB/SmolLM3-3B",
    messages=[{"role": "user", "content": prompt}],
    max_tokens=100
)

print("Generated Output:")
print(response.choices[0].message.content)
```

OUTPUT

```
Enter your prompt: What is AI?
Generated Output:
Artificial Intelligence is a branch of computer science
that enables machines to perform tasks requiring human intelligence.
```

9. Python Code Generation using Structured Prompting Design structured prompts to instruct an LLM to generate Python programs for specified computational problems and validate the generated code.

CODE

```
!pip install -q google-genai
```

```
from google.colab import userdata
from google import genai

client = genai.Client(api_key=userdata.get("GEMINI_API_KEY"))

problem = input("Enter the programming problem: ")

prompt = f"""
Generate a Python program for this problem: {problem}
Requirements:
- Use simple Python.
- Return executable code only.
- Avoid unnecessary explanations.
"""

response = client.models.generate_content(
    model="gemini-2.5-flash",
    contents=prompt
)

code = response.text.replace("```python", "").replace("```", "").strip()

print("Generated Code:")
print(code)

try:
    compile(code, "<generated_code>", "exec")
    print("\nSyntax Validation: Successful")
```

```
except SyntaxError as e:
    print("\nSyntax Error:", e)
```

OUTPUT

Generated Code:

```
number = int(input("Enter a number: "))
if number % 2 == 0:
    print("Even")
else:
    print("Odd")
```

Syntax Validation: Successful

10. SQL Query Generation using Structured Prompting Design structured prompts that provide database schema, table details, and requirements to an LLM and generate appropriate SQL queries.

CODE

```
!pip install -q google-genai

from google.colab import userdata
from google import genai

client = genai.Client(api_key=userdata.get("GEMINI_API_KEY"))

schema = """
Table: Students
Columns: id, name, department, marks
"""

requirement = input("Enter your SQL requirement: ")

prompt = f"""
```

Generate an SQL query for the following database.

Schema:

{schema}

Requirement:

{requirement}

Return only the SQL query.

"""

```
response = client.models.generate_content(
    model="gemini-2.5-flash",
    contents=prompt
)

print("Generated SQL Query:")
print(response.text)
```

OUTPUT

SELECT * FROM Students WHERE marks > 80;

11. Comparative Analysis of Zero-shot, One-shot and Few-shot Prompting Generate responses for the same task using zero-shot, one-shot, and few-shot prompts and compare the quality, relevance, accuracy, and consistency of the responses.

CODE

```
!pip install -q google-genai

from google.colab import userdata
from google import genai
```

```
client = genai.Client(api_key=userdata.get("GEMINI_API_KEY"))

task = "Classify the sentiment: The product is excellent."

prompts = {
    "Zero-shot": task,
    "One-shot": """
    Example: The service is bad -> Negative
    Classify: The product is excellent.
    """,
    "Few-shot": """
    Example: The movie is good -> Positive
    Example: The food is terrible -> Negative
    Example: The phone is average -> Neutral
    Classify: The product is excellent.
    """
}

for name, prompt in prompts.items():
    response = client.models.generate_content(
        model="gemini-2.5-flash",
        contents=prompt
    )
    print(name + " Output:")
    print(response.text)
    print()
```

OUTPUT

Zero-shot Output:

Positive

One-shot Output:

Positive

Few-shot Output:

Positive

12. Prompt Evaluation, Comparison and Refinement using LLM Responses Design multiple prompts for a given text-generation task, evaluate their outputs usingpredefined criteria such as relevance, accuracy, completeness, clarity, and format adherence, and refine the best-performing prompt.

CODE

```
!pip install -q google-genai


from google.colab import userdata
from google import genai


client = genai.Client(api_key=userdata.get("GEMINI_API_KEY"))


prompts = [
    "Explain Artificial Intelligence.",
    "Explain Artificial Intelligence in simple words.",
    "Explain Artificial Intelligence in simple words with two examples and applications."
]


for i, prompt in enumerate(prompts, 1):
    response = client.models.generate_content(
        model="gemini-2.5-flash",
        contents=prompt
    )
```

```
    print(f"\nPrompt {i}: {prompt}")
    print("Response:", response.text)

evaluation_prompt = f"""
Evaluate these three prompts for explaining Artificial Intelligence:
{prompts}

Score each prompt from 1 to 5 based on relevance,
accuracy, completeness, clarity, and format adherence.

Choose the best prompt and suggest an improved version.
Return the evaluation in a simple table.
"""

evaluation = client.models.generate_content(
    model="gemini-2.5-flash",
    contents=evaluation_prompt
)

print("\nPrompt Evaluation:")
print(evaluation.text)
```

OUTPUT

```
Prompt Evaluation:

Prompt 1: Score 3/5
Prompt 2: Score 4/5
Prompt 3: Score 5/5
```

Best Prompt:

Explain Artificial Intelligence in simple words with two examples and applications.